

Assignment 1 - Mini Project

Console-Based Record Management Application Using Python

Project Name:
Student Record Management System

Name:-Sara Sachin Wadke

Rollno: 41

***Subject: Programming with Python
& Relational Database***

Index

Sr.no	Topic	Pg.no
1	Project Introduction	3
2	Objectives	3
3	Features	4
4	Python Concepts Used in single application	4
5	Files used	6
6	How the Application Works	7
7	Exception Handling and Validation	7
8	Working Demonstration / Screenshots	8
9	Testing	14
10	Conclusion	15
11	GitHub Repository	15

1. Project Introduction

The **Console-Based Record Management Application Using Python** is a text-based, menu-driven application developed using Python. The application runs directly in the console/command prompt and allows the user to manage records through numbered menu options.

For this project, student records have been selected as the type of records to be managed. The user can perform operations such as adding, viewing, searching, updating, and deleting student records through the console.

The application uses a JSON file for storing records. This allows the data to remain available even after the console application is closed and started again.

2. Objectives

- To develop a fully working console-based record-management application using Python.
- To implement a text-based menu-driven interface for managing records.
- To demonstrate Python data types, variables, conditional statements, loops, and functions.
- To implement exception handling for invalid inputs and file-related errors.
- To use File I/O for permanent storage of records.
- To perform basic record-management operations through the console.
- To create a readable, structured, and executable Python application

3. Features

- Add Student Record
- View All Records
- Search Student by Student ID
- Update Student Record
- Delete Student Record with confirmation
- Input validation and exception handling
- JSON file storage for data persistence
- Menu-driven console interface

4. Python Concepts Used in Single Application

Python Concept	How It Is Used	Where / Code Used in <code>student_record_manager.py</code>
Variables	Used to store values such as file name, student details, menu choice, and temporary values.	<code>FILE_NAME = "students.json";</code> <code>student_id = ...; name = ...;</code> <code>choice = input(...)</code>
Data Types	Different data types are used according to the type of information being stored.	<code>age = int(input(...)) → Integer;</code> <code>percentage = float(input(...)) → Float;</code> <code>name, course, email → String</code>
List	Used to store multiple student records together.	<code>records = load_records();</code> <code>records.append(student)</code>

Dictionary	Used to store the details of one student using key-value pairs.	<code>student = {"student_id": student_id, "name": name, "age": age, ...}</code>
Conditional Statements	Used to make decisions, validate input, and control menu operations.	<code>if choice == "1"; elif choice == "2"; else;;</code> also used in age and percentage validation
Loops	Used to repeatedly display the console menu and process records.	<code>while True:</code> in <code>main()</code> ; for <code>student</code> in <code>records</code> ; in <code>Add</code> , <code>Search</code> , <code>Update</code> , and <code>Delete</code> operations
Functions	The application is divided into separate functions for different tasks, making the code organized and reusable.	<code>def add_record(records); def view_records(records); def search_record(records); def update_record(records); def delete_record(records);</code>
Exception Handling	Used to handle invalid input and file-related errors without stopping the application unexpectedly.	<code>try: / except ValueError;; except FileNotFoundError; except json.JSONDecodeError; except OSError</code>
File I/O	Used to read existing records and save changes to the JSON file.	<code>with open(FILE_NAME, "r") as file;; with open(FILE_NAME, "w") as file:</code>
JSON	Used to store student records in JSON format.	<code>json.load(file)</code> reads records; <code>json.dump(records, file, indent=4)</code> saves records

String Handling	Used to clean and process input entered through the console.	<code>.strip()</code> removes extra spaces; <code>.lower()</code> converts confirmation input to lowercase
Input Validation	Used to ensure that users enter acceptable values.	<code>if age <= 0; elif age > 100; if percentage < 0 or percentage > 100; if value == ""</code>
Menu-Driven Console Application	The user interacts with the application through a text-based console menu and numbered choices.	<code>choice = input("\n Enter your choice (1-6): ").strip()</code> followed by <code>if/elif</code> menu conditions
CRUD Operations	Used to perform Create, Read, Update, and Delete operations on records.	Create → <code>add_record()</code> ; Read → <code>view_records()</code> / <code>search_record()</code> ; Update → <code>update_record()</code> ; Delete → <code>delete_record()</code>

5. Files Used

- `student_record_manager.py` - Main Python source code.
- `students.json` - Stores student records in JSON format.
- `README.md` - Project information and instructions.
- Screenshots - Screenshots showing the application working.
- `Assignment1.pdf` - Complete assignment documentation.

6. How the Application Works

1. Run `student_record_manager.py` from the terminal or command prompt.
2. The application loads existing records from `students.json`.
3. A text-based main menu is displayed in the console.
4. The user enters a menu number from 1 to 6.
5. The selected operation is executed in the console.
6. The user enters or views the required record information.
7. New, updated, or deleted records are saved to `students.json`.
8. The menu appears again so the user can perform another operation.
9. Selecting 6 - Exit Application closes the console application.

7. Exception Handling and Validation

Since the application receives information directly from the user through the console, input validation and exception handling are implemented to prevent common errors.

- Age must be entered as a valid whole number.
- Age must be greater than 0 and within the allowed range.
- Percentage must be a number between 0 and 100.
- Required text fields cannot be left empty.
- Duplicate Student IDs are not allowed.
- Invalid menu choices are handled with an error message.
- `try-except` blocks handle invalid numeric input.
- File-related errors are handled safely.
- Missing or invalid JSON data is handled using exception handling.

8. Working Demonstration / Screenshots

The following screenshots demonstrate the major operations of the application. The same screenshots are also maintained in the project's screenshots folder.

- **Main menu**

```
PS C:\Users\Sara Wadke\Downloads\Student_Record_Management_System> python student_record_manager.py
```

```
=====
                        STUDENT RECORD MANAGEMENT SYSTEM
=====

                        MAIN MENU
-----
1. Add Student Record
2. View All Records
3. Search Student
4. Update Student Record
5. Delete Student Record
6. Exit Application
-----

Total Records: 0
-----

Enter your choice (1-6): █
```

- **Add record**

```
=====
                        ADD STUDENT RECORD
=====

Student ID       : 101
Student Name     : sara wadke
Age              : 20
Course           : MCA
Email            : sarawadke2018@gmail.com
Percentage       : 95
-----

[SUCCESS] Student record added successfully.
-----

Press Enter to continue...█
```

- **View records**

VIEW ALL STUDENT RECORDS

Total Records: 3

Record 1

Student ID : 101
Student Name : sara wadke
Age : 20
Course : MCA
Email : sarawadke2018@gmail.com
Percentage : 95.00%

Record 2

Student ID : 102
Student Name : prisha
Age : 21
Course : BSCIT
Email : prisha@gmail.com
Percentage : 89.00%

Record 3

Student ID : 103
Student Name : siddhesh
Age : 20
Course : BMS
Email : siddhesh@gmail.com
Percentage : 70.00%

- **Search student**

Enter your choice (1-6): 3

```
=====
                        SEARCH STUDENT
=====
Enter Student ID : 101

-----
                        STUDENT DETAILS
-----
Student ID       : 101
Student Name     : sara wadke
Age              : 20
Course           : MCA
Email            : sarawadke2018@gmail.com
Percentage       : 95.00%
-----
```

- **Update record**

Enter your choice (1-6): 4

```
=====
                        UPDATE STUDENT RECORD
=====
Enter Student ID : 102

Current Record
-----
Name              : prisha
Age               : 21
Course            : BSCIT
Email             : prisha@gmail.com
Percentage        : 89.00%
-----

Enter the new details:
Student Name      : prisha wadke
Age               : 20
Course            : BSCIT
Email             : prishawadke@gmail.com
Percentage        : 90

-----
[SUCCESS] Student record updated successfully.
-----

Press Enter to continue...█
```

- **Delete record**

Enter your choice (1-6): 5

```
=====
                        DELETE STUDENT RECORD
=====
Enter Student ID : 103

-----
Student Name      : siddhesh
Student ID       : 103
-----
Are you sure you want to delete this record? (y/n): y

-----
[SUCCESS] Student record deleted successfully.
-----
```

- **exception handling - Invalid input handling**

Enter your choice (1-6): 1

```
=====
                        ADD STUDENT RECORD
=====
Student ID       : 103
Student Name     : smita
Age              : abc
[ERROR] Age must be a whole number.
Age              : 21
Course           : BCA
Email            : smita@gmail.com
Percentage       : 90

-----
[SUCCESS] Student record added successfully.
-----
```

- **file persistence - Records retained after restarting the application**

PS C:\Users\Sara Wadke\Downloads\Student_Record_Management_System> python student_record_manager.py

```
=====
STUDENT RECORD MANAGEMENT SYSTEM
=====
```

MAIN MENU

- ```

1. Add Student Record
2. View All Records
3. Search Student
4. Update Student Record
5. Delete Student Record
6. Exit Application

```

Total Records: 3

Enter your choice (1-6): 2

```
=====
VIEW ALL STUDENT RECORDS
=====
```

Total Records: 3

```

Record 1

```

```
Student ID : 101
Student Name : sara wadke
Age : 20
Course : MCA
Email : sarawadke2018@gmail.com
Percentage : 95.00%

```

```

Record 2

```

```
Student ID : 102
Student Name : prisha wadke
Age : 20
Course : BSCIT
Email : prishawadke@gmail.com
Percentage : 90.00%

```

```

Record 3

```

```
Student ID : 103
Student Name : smita
Age : 21
Course : BCA
Email : smita@gmail.com
Percentage : 90.00%

```

---

- JSON file containing stored student records

```
{ } students.json > ...
```

```
1 [
2 {
3 "student_id": "101",
4 "name": "sara wadke",
5 "age": 20,
6 "course": "MCA",
7 "email": "sarawadke2018@gmail.com",
8 "percentage": 95.0
9 },
10 {
11 "student_id": "102",
12 "name": "prisha wadke",
13 "age": 20,
14 "course": "BSCIT",
15 "email": "prishawadke@gmail.com",
16 "percentage": 90.0
17 },
18 {
19 "student_id": "103",
20 "name": "smiya",
21 "age": 21,
22 "course": "BCA",
23 "email": "smiya@gmail.com",
24 "percentage": 90.0
25 }
26]
```

## 9. Testing

| Test Case                      | Result            |
|--------------------------------|-------------------|
| Add valid student record       | Passed            |
| View all records               | Passed            |
| Search existing student        | Passed            |
| Update existing student        | Passed            |
| Delete student record          | Passed            |
| Invalid age input              | Handled correctly |
| Invalid percentage input       | Handled correctly |
| Data persistence after restart | Passed            |

## 10. Conclusion

The Console-Based Record Management Application Using Python successfully demonstrates the required Python programming concepts through a functional Student Record Management System.

The application provides a text-based console interface through which users can add, view, search, update, and delete records. It uses functions to organize the program, conditional statements and loops for menu control, exception handling for errors, and JSON File I/O for permanent data storage.

The application therefore demonstrates the major requirements of a console-based record-management application using Python in a simple and functional manner.

## 11. GitHub Repository

The complete project is uploaded to a GitHub repository containing the Python source code, JSON data file, README.md, assignment documentation, and screenshots.

[https://github.com/sara-wadke/Assignment\\_1\\_Console\\_Record\\_Management\\_Application.git](https://github.com/sara-wadke/Assignment_1_Console_Record_Management_Application.git)